

Homework-03

Cassidy Thorpe

2026-05-26

Set Up

```
# load in packages
library(tidyverse)
library(janitor)
library(readxl)
library(here)
library(ggeffects)
```

```
# store kelp data as an object called kelp
kelp <- read_csv(here("data", "temp-kelp.csv"))
# store egg data as an object called egg_data
egg_data <- read_csv(here("data", "personal-data-project - Sheet1.csv"))
```

Problems

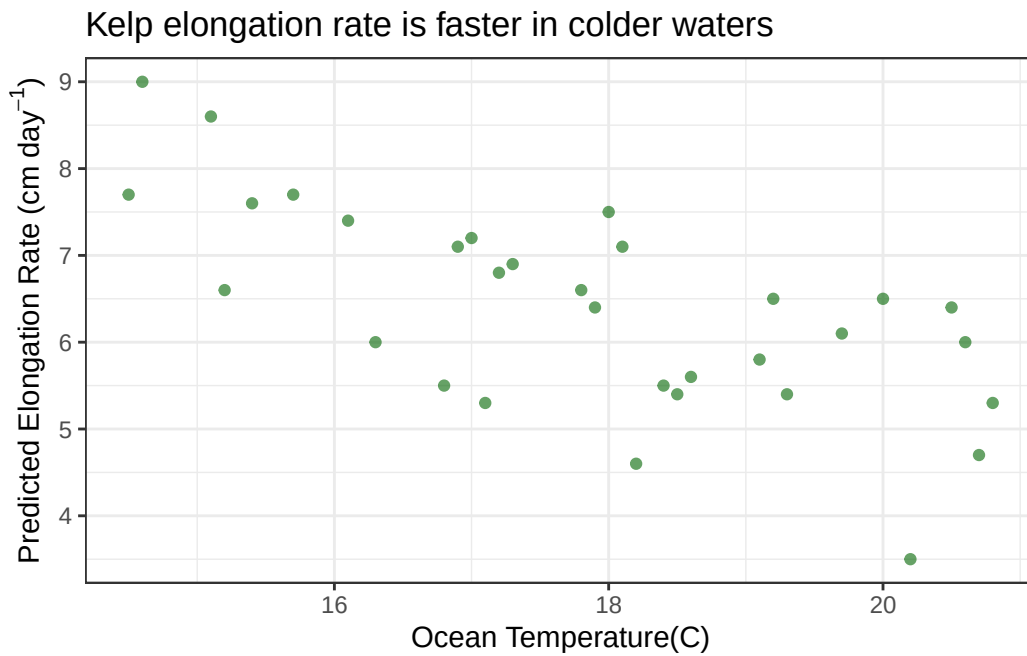
Problem 1: Giant Kelp Fronds

a. An appropriate test

To determine the strength of the relationship between temperature and elongation rate, you could do a Pearson's correlation and a Spearman rank correlation. A Pearson's correlation is parametric and is best for understanding the strength of the correlation between variables when there is a linear relationship between continuous variables. A Spearman's rank correlation is non-parametric and useful for understanding the strength of the correlation between variables that show a monotonic relationship between variables.

b. Create a visualization

```
# base layer: ggplot
ggplot(data = kelp, # use kelp data frame
       aes(x = temp_c, # x-axis: temperature (C)
           y = kelp_elong)) + # y-axis: kelp elongation rate (cm/day)
# 1st layer: show raw data points
geom_point(color = "darkgreen", # points = dark green
           alpha = 0.6) + # points = slightly transparent
# label and y axes
labs(x = "Ocean Temperature(C)",
     y = expression(paste("Predicted Elongation Rate (cm ", day-1,"))),
# add title
     title = "Kelp elongation rate is faster in colder waters") +
# change theme
theme_bw() +
# remove legend
theme(legend.position = "none")
```



c. Check assumptions and run test

Create Linear Model

```
# create new object called kelp model to store linear model
# function: lm
# formula: dependent ~ independent, data = data frame
kelp_model <- lm(kelp_elong ~ temp_c, data = kelp)
```

```
# show summary of linear model
summary(kelp_model)
```

Call:

```
lm(formula = kelp_elong ~ temp_c, data = kelp)
```

Residuals:

Min	1Q	Median	3Q	Max
-1.8643	-0.5017	0.1790	0.5659	1.2177

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	14.08645	1.49219	9.440	1.72e-10 ***
temp_c	-0.43179	0.08321	-5.189	1.37e-05 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.8665 on 30 degrees of freedom

Multiple R-squared: 0.473, Adjusted R-squared: 0.4554

F-statistic: 26.93 on 1 and 30 DF, p-value: 1.366e-05

```
# show in a table
flextable::as_flextable(kelp_model)
```

	Estimate	Standard Error	t value	Pr(> t)
(Intercept)	14.086	1.492	9.440	0.0000 ***
temp_c	-0.432	0.083	-5.189	0.0000 ***

Estimate	Standard Error	t value	Pr(> t)
Signif. codes: 0 <= '***' < 0.001 < '**' < 0.01 < '*' < 0.05			

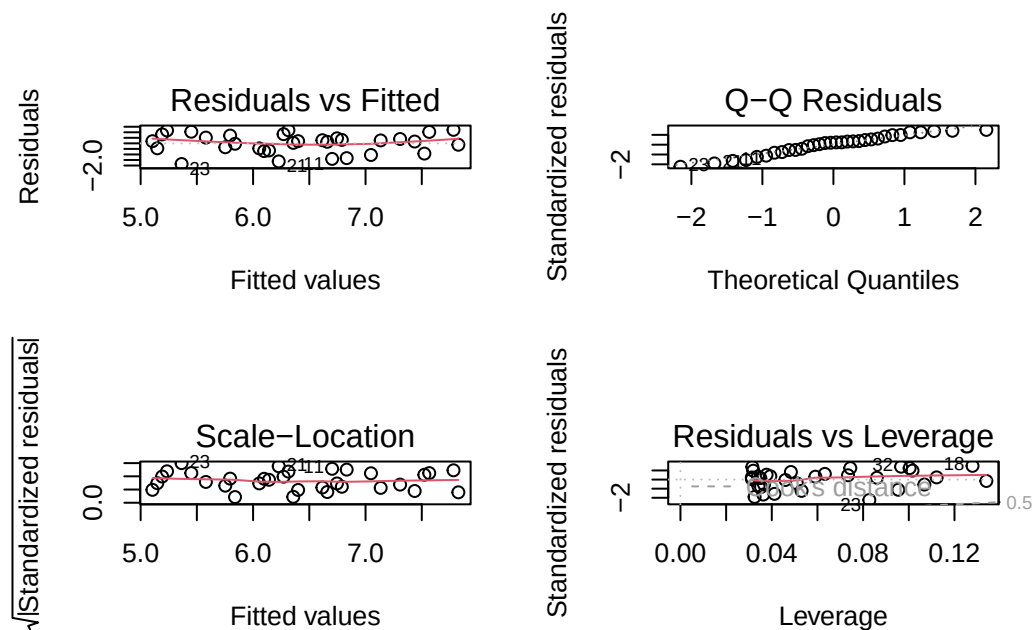
Residual standard error: 0.8665 on 30 degrees of freedom

Multiple R-squared: 0.473, Adjusted R-squared: 0.4554

F-statistic: 26.93 on 30 and 1 DF, p-value: 0.0000

Check Assumptions

```
# create 2x2 grid to show diagnostic plot
par(mfrow = c(2,2))
# show diagnostic plots
plot(kelp_model)
```



The QQ residuals plot shows the data points following the line. This suggests the distribution of the residuals is normal. The residuals vs. fitted and scale locations plots don't seem to show a pattern. The data seems to be randomly and evenly distributed which indicates that the residuals are homoscedastic. The residuals vs leverage plot does not show points outside of the dashed lines which indicates that there are no outliers influencing the model.

Run Test

```
# run pearson's correlation test
# # formula: (data set$independent, data set$dependent)
kelp_corp <- cor.test(kelp$temp_c, kelp$kelp_elong, method = "pearson")
# show output
show(kelp_corp)
```

Pearson's product-moment correlation

```
data: kelp$temp_c and kelp$kelp_elong
t = -5.189, df = 30, p-value = 1.366e-05
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.8359665 -0.4460157
sample estimates:
      cor
-0.6877489
```

d. Results communication

To evaluate the strength of the relationship between temperature and kelp frond elongation, I used Pearson's correlation (r). This test is a good choice because both ocean temperature and elongation rate are continuous variables, and the diagnostic checks show that the data satisfies the assumptions of normality and linearity. I suspect a linear relationship between ocean temperature and kelp frond elongation rate

I found a strong negative relationship between ocean temperature and giant kelp frond elongation rate (Pearson's $r = -0.7$, $t(30) = -5.189$, $p < 0.001$, $\alpha = 0.05$).

e.

I would tell them that ocean temperature and the growth rate of kelp are related and that kelp fronds in high temperature ocean conditions tend to have a slower growth rate. I would also tell them that more tests need to be done in order to fully describe the relationship between temperature and growth rate.

f. Double check your own work

```
# run spearman's rank test
kelp_cors <- cor.test(kelp$temp_c, kelp$kelp_elong, method = "spearman")
# show output
show(kelp_cors)
```

Spearman's rank correlation rho

```
data: kelp$temp_c and kelp$kelp_elong
S = 9216.1, p-value = 1.29e-05
alternative hypothesis: true rho is not equal to 0
sample estimates:
      rho
-0.6891684
```

The Spearman's rank correlation would have led me to make the same decision - that there is a relationship between ocean temperature and kelp frond elongation rate.

We found a strong negative relationship between ocean temperature and giant kelp frond elongation rate (Spearman $\rho = -0.69$, $S = 9216.1$, $p < 0.001$, $\alpha = 0.05$).

Spearman rank correlation is the non-parametric version of the Pearson's correlation. The Spearman test is better suited to variables that are monotonic and it converts the data in to ranks instead of comparing the raw data like Pearson's r does.

Problem 2: Personal Data

```
# create new object from egg_data
egg_data_clean <- egg_data |>
# clean names
clean_names() |>
# remove any row where all values are NA
remove_empty(which = "rows") |>
# remove the empty (1st) column that was all NA
select(-1) |>
# change names from 1, 2, 3, etc.
set_names("date", "number_eggs_eaten", "servings_yogurt_eaten",
```

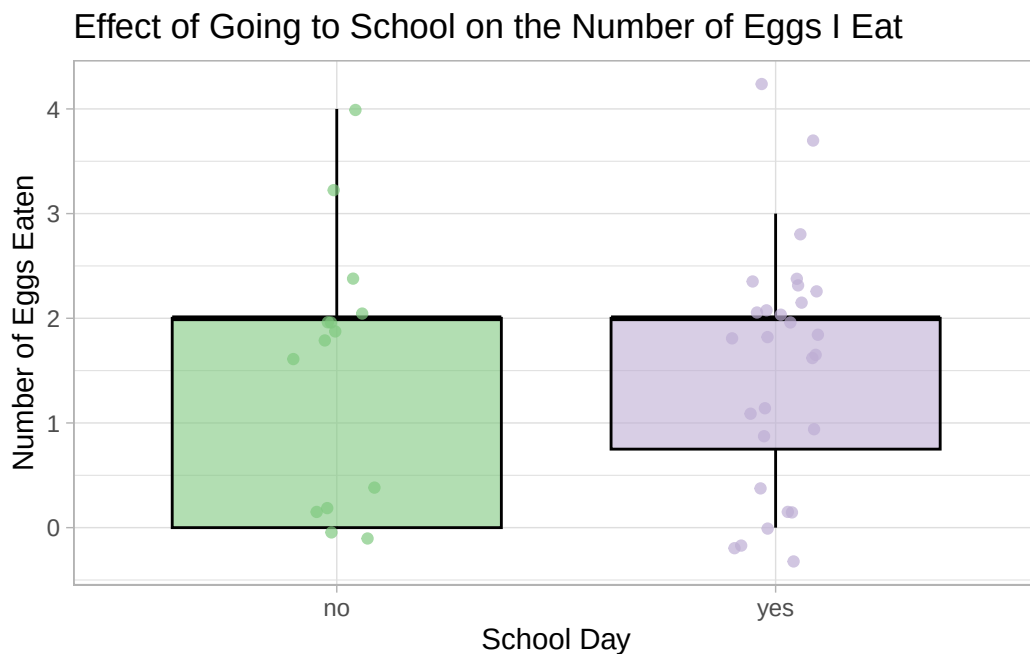
```

"breakfast_at_school", "servings_meat_eaten", "school_day") |>
# remove the first row (old spreadsheet cloumn titles)
slice(-1) |>
# convert the columns from text to numbers
mutate(number_eggs_eaten = as.numeric(number_eggs_eaten),
       servings_yogurt_eaten = as.numeric(servings_yogurt_eaten),
       servings_meat_eaten = as.numeric(servings_meat_eaten))

# base layer: ggplot
# use clean data set
# set x-axis to school day
# set y-axis to number of eggs eaten
# make the point color different based on school day
# make the fill color of the box plot different based on school day
ggplot(data = egg_data_clean,
      aes(x = school_day,
          y = number_eggs_eaten,
          color = school_day,
          fill = school_day)) +
# 1st layer: geom_boxplot
# make the boxplot fill more transparent
# make the lines of the boxes black
# hide the boxplot outlier
geom_boxplot(alpha = 0.6,
             color = "black",
             outlier.shape = NA) +
# 2nd layer: geom_jitter
# make the individual dots visible but still within their category (0.1)
# make the points transparent enough to see if they overlap
geom_jitter(width = 0.1,
            alpha = 0.7) +
# change boxplot color
# change point color
scale_fill_brewer(palette = "Accent") +
scale_color_brewer(palette = "Accent") +
# label x and y axes
# add title
labs(x = "School Day",
     y = "Number of Eggs Eaten",
     title = "Effect of Going to School on the Number of Eggs I Eat") +
# change the theme
theme_light() +

```

```
# remove the legend
theme(legend.position = "none")
```



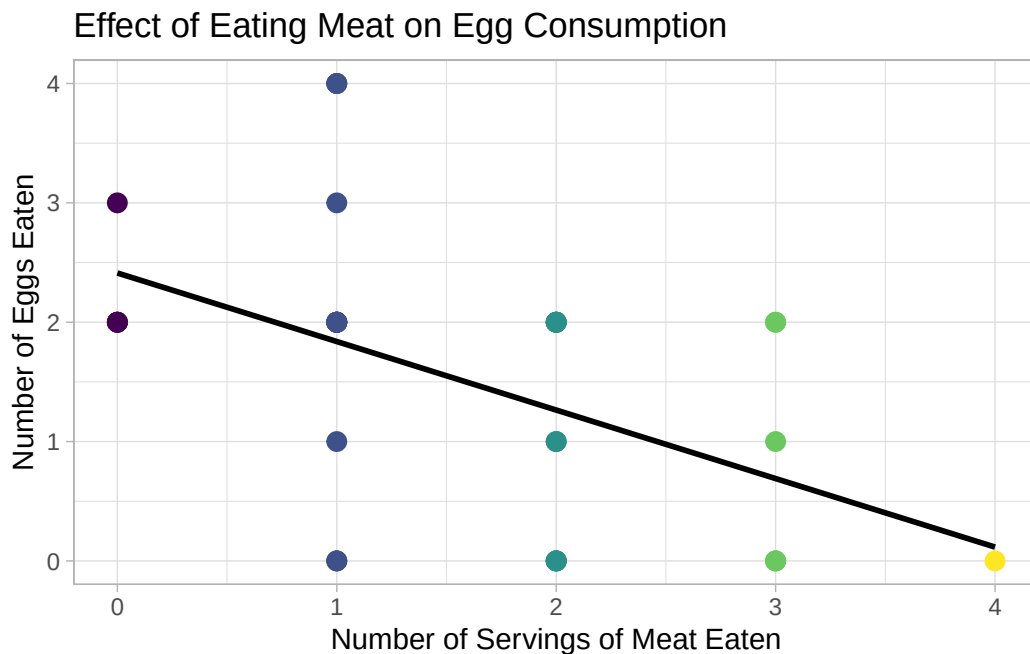
```
# base layer: ggplot
# x-axis: meat consumption
# y-axis: egg consumption
ggplot(data = egg_data_clean,
       aes(x = servings_meat_eaten,
           y = number_eggs_eaten))+
  # 1st layer: geom_point for a scatterplot
  # color points by the number of servings of meat eaten
  # make the points size 3
  geom_point(aes(color = servings_meat_eaten),
            size = 3) +
  # 2nd layer: geom_smooth for a trend line
  # tell geom_smooth to make a straight line, not a squiggle
  # make the line black
  # don't show the standard error
  geom_smooth(method = "lm",
             color = "black",
             se = FALSE) +
  # change color scheme for continuous variables
```



```

scale_color_viridis_c() +
# label x and y axes
# add title
labs(x = "Number of Servings of Meat Eaten",
     y = "Number of Eggs Eaten",
     title = "Effect of Eating Meat on Egg Consumption") +
# change theme
theme_light() +
# remove legend
theme(legend.position = "none")

```



```

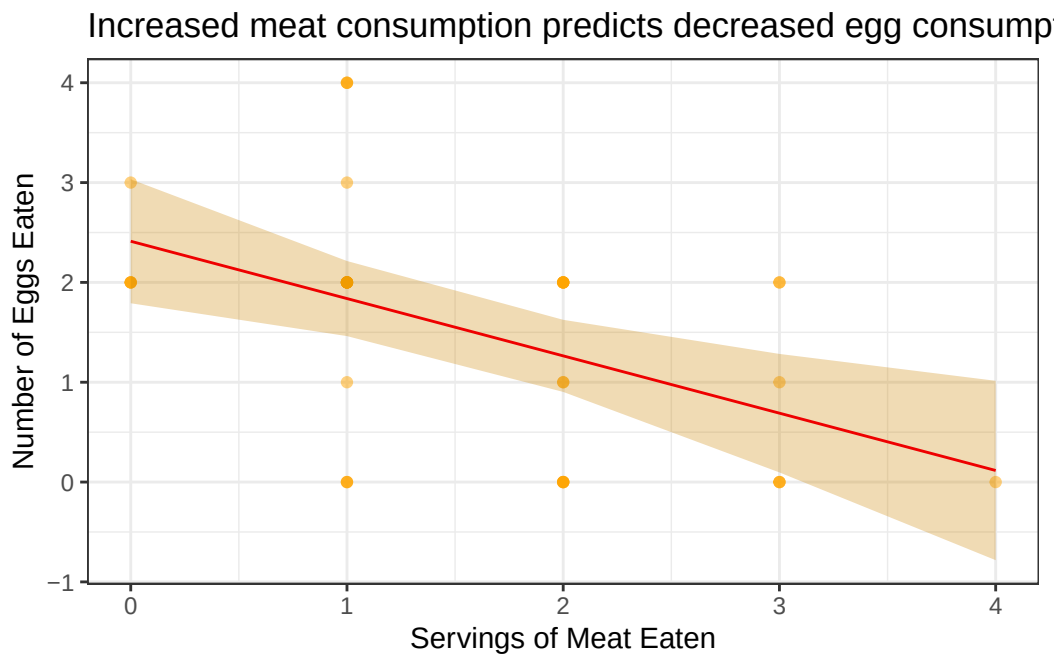
egg_model <- lm(number_eggs_eaten ~ servings_meat_eaten,
               data = egg_data_clean)
# create new object to store predictions of the model
egg_predict <- ggpredict(egg_model, terms = "servings_meat_eaten")
# base layer: ggplot
ggplot(data = egg_predict, # use predictions data frame
       aes(x = x, # x-axis: x
           y = predicted # y-axis: predicted
       )) +
# 1st layer: geom_point to show raw data points
geom_point(data = egg_data_clean,

```

```

aes(x = servings_meat_eaten,
    y = number_eggs_eaten), #
color = "orange", # point color = dark green
alpha = 0.5) + # make points slightly transparent
# 2nd layer: ribbon to show 95% CI
geom_ribbon(aes(ymin = conf.low, # lower bound of 95% CI
              ymax = conf.high), # upper bound of 95% CI
           fill = "orange3",
           alpha = 0.3) + # make ribbon transparent
# 3rd layer: line representing model predictions
geom_line(color = "red2") +
# relabel x and y axes and add title
labs(x = "Servings of Meat Eaten",
     y = "Number of Eggs Eaten",
     title = "Increased meat consumption predicts decreased egg consumption") +
# change theme
theme_bw()

```



```

# show summary of linear model
summary(egg_model)

```

```
Call:
lm(formula = number_eggs_eaten ~ servings_meat_eaten, data = egg_data_clean)
```

Residuals:

```
      Min       1Q   Median       3Q      Max
-1.8382 -0.6900  0.1618  0.7359  2.1618
```

Coefficients:

```
              Estimate Std. Error t value Pr(>|t|)
(Intercept)      2.4123     0.3072   7.851 1.26e-09 ***
servings_meat_eaten -0.5741     0.1689  -3.399  0.00154 **
---
```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.047 on 40 degrees of freedom

Multiple R-squared: 0.2241, Adjusted R-squared: 0.2047

F-statistic: 11.55 on 1 and 40 DF, p-value: 0.001543

```
# show in a table
flextable::as_flextable(egg_model)
```

	Estimate	Standard Error	t value	Pr(> t)
(Intercept)	2.412	0.307	7.851	0.0000 ***
servings_meat_eaten	-0.574	0.169	-3.399	0.0015 **

Signif. codes: 0 <= '***' < 0.001 < '**' < 0.01 < '*' < 0.05

Residual standard error: 1.047 on 40 degrees of freedom

Multiple R-squared: 0.2241, Adjusted R-squared: 0.2047

F-statistic: 11.55 on 40 and 1 DF, p-value: 0.0015

Check Assumptions

```
# create 2x2 grid to show diagnostic plot
par(mfrow = c(2,2))
# show diagnostic plots
plot(egg_model)
```

